IBM Developer
SKILLS NETWORK

# Winning Space Race with Data Science

Bharath Kumar Mohan
April 25 2026

# Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

# Executive Summary

- Summary of methodologies
- Summary of all results

# Introduction

- Project background and context
- Problems you want to find answers

Section 1

# Methodology

# Methodology

## Executive Summary

- Data collection methodology:
  - SpaceX API (RESTful API Calls) - Source: Official SpaceX API (api.spacexdata.com/v4/)
  - Web Scraping (BeautifulSoup) - Source: Wikipedia's "List of Falcon 9 and Falcon Heavy launches" page
- Perform data wrangling
  - Data wrangling transformed raw launch data into a structured, analysis-ready dataset by loading data into a Pandas DataFrame, imputing missing and null values and categorize data into 3 different launch sites
- Perform exploratory data analysis (EDA) using visualization and SQL
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
- Split data → Standardize features → Train model with GridSearchCV for hyperparameter tuning → Evaluate using confusion matrix, accuracy, precision, recall, and ROC-AUC score.

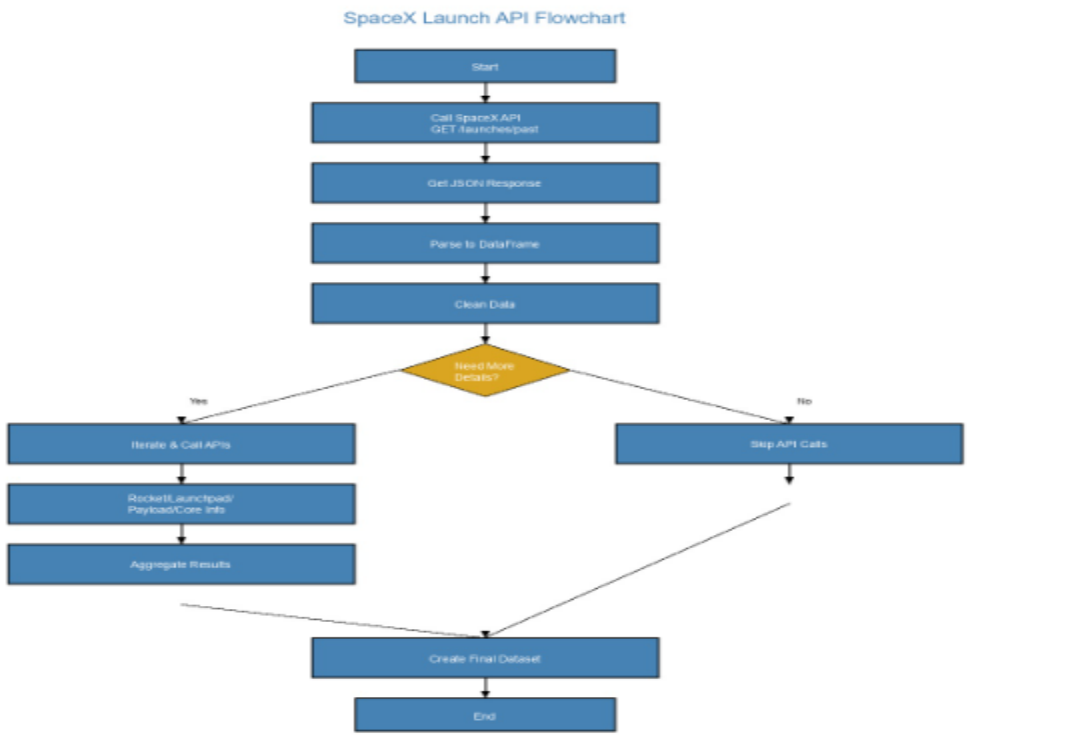
# Data Collection

- Describe how data sets were collected.
- You need to present your data collection process use key phrases and flowcharts

# Data Collection – SpaceX API

**Main Steps**:

- **Call API** - GET /launches/past endpoint
- **Parse Response** - Convert JSON to DataFrame
- **Clean Data** - Filter and prepare data
- **Decision Point** - Fetch additional details or proceed
- **Enrichment Calls** (if needed):
  - Rocket information
  - Launchpad details
  - Payload data
  - Core information
- **Aggregate** - Combine all data
- **Create Final Dataset** - Ready for analysis



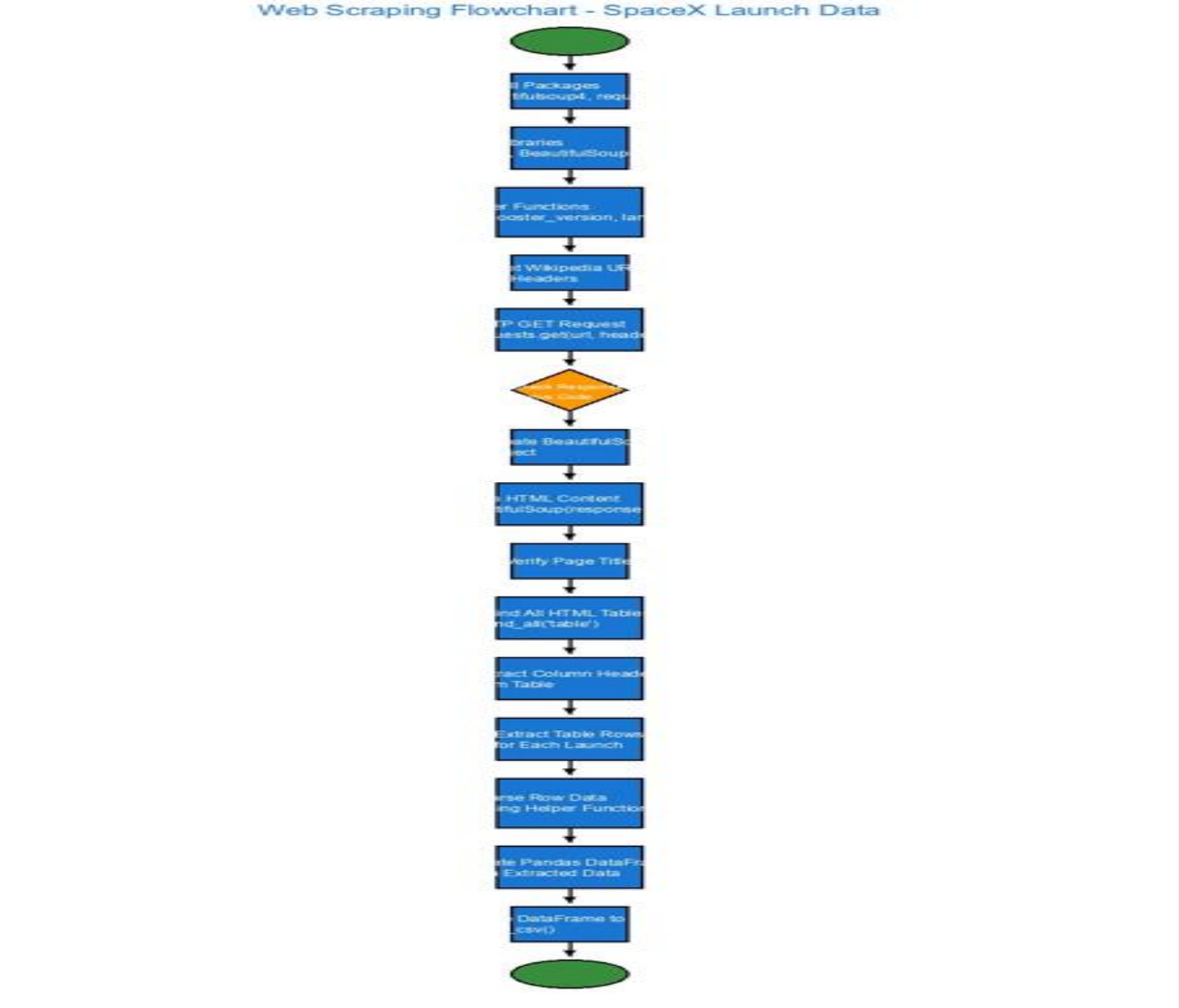

**GitHub URL:**

[Data Collection - SpaceX API Notebook]

# Data Collection - Scraping

| Step | Purpose |
|---|---|
| **1. Send HTTP request** | Retrieve raw HTML from the webpage. |
| **2. Parse HTML with Beautiful Soup** | Convert HTML into a structured parse tree. |
| **3. Inspect HTML structure** | Identify tags, classes, and patterns for extraction. |
| **4. Extract data** | Use `.find()`, `.find_all()`, attributes, and text extraction. |
| **5. Clean/store data** | Prepare data for analysis or storage. |

[Web Scraping Git Hub repo]



# Data Wrangling

- **Phase 1: Data Loading & Inspection**

  *Exploratory Data Analysis (EDA)*

- **Phase 2: Data Quality Assessment**

  *Identify data characteristics*

- **Phase 3: Categorical Analysis**

  *Value counts for distribution analysis*

- **Phase 4: Mission Outcome Categorization**

  *Landing outcome classification*

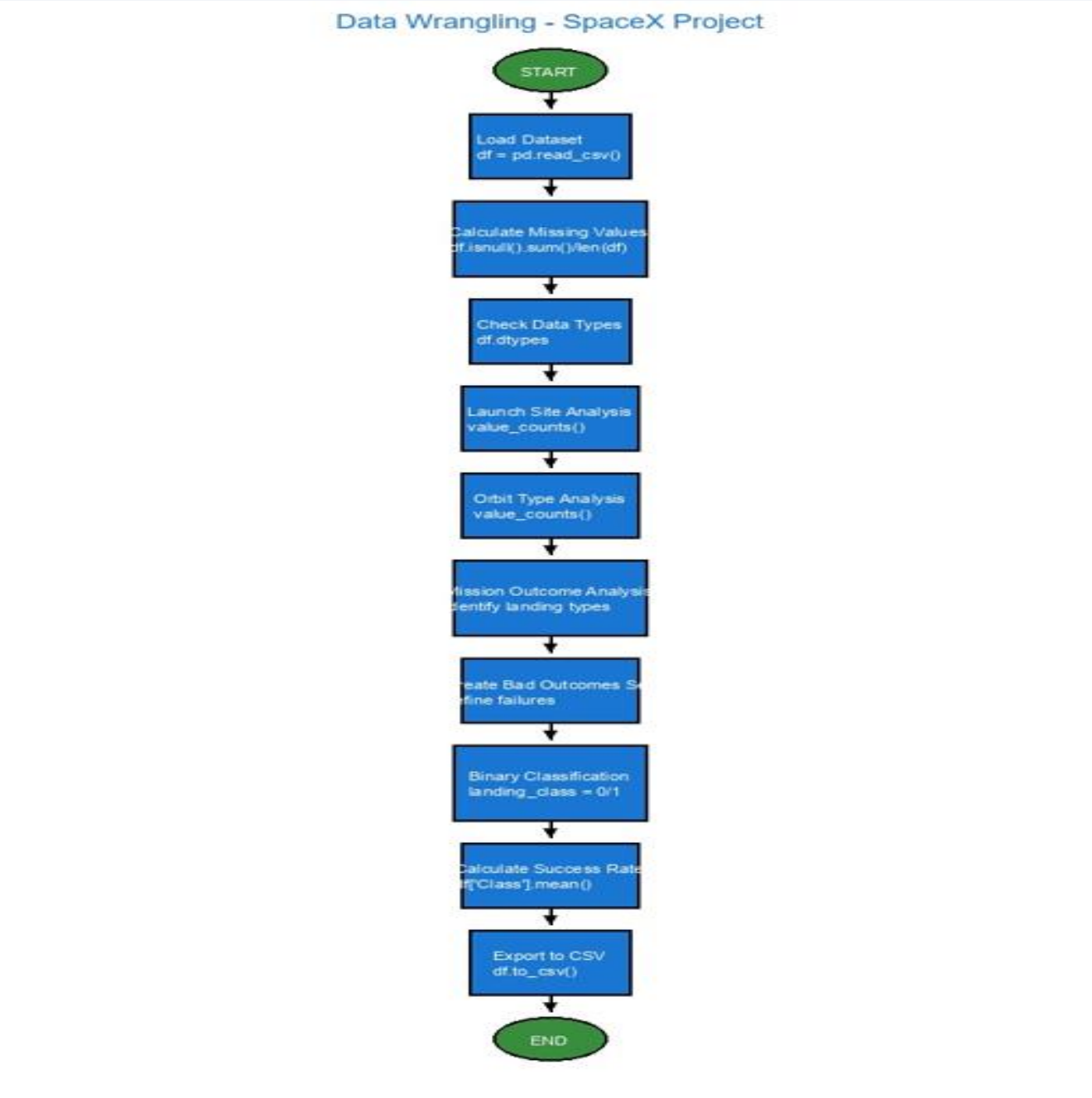
- **Phase 5: Feature Engineering - Binary Label Creation**

  *Convert outcomes into training labels*

- **Phase 6: Success Rate Calculation & Export**

  *Prepare data for machine learning*

- [Data Wrangling - Notebook in git hub]



# EDA with Data Visualization

Charts plotted when studying EDA

- Scatter Chart
- Bar Chart
- Line Chart
- Box Plot
- Cat Plot
- Pie chart
- Histogram Chart

[Git hub - EDA Data Visualization]

# EDA with SQL

SQL queries performed

- Select Distinct (to display unique Lauch Sites)
- Select with limit (to display top 5 records)
- Select with Group By and Having Clauses (display sum of payload grouped by booster)
- Select with min function (to display the first successful landing date)
- Select with inline queries (using two alias table for success and failure mission outcomes)
- Max and Count functions using Group By clause
- Substr Function (For date extraction)

- Git hub repo - EDA with sql queries magic functions

# Build an Interactive Map with Folium

- List of Map Objects with Folium
  i. **Folium.Map (Base Map Object)**
  ii. **Folium.Circle**
  iii. **Folium.Marker (Marker Cluster object added to marker)**
  iv. **Folium.PolyLine (to calculate distance and mouse position widget added to the Lat/Lon coordinate)**

***These objects collectively create an interactive geographic analysis to identify patterns between launch site locations and success rates, helping determine optimal factors for future launch site selection.***

Git Hub repo -Folium Map library

# Build a Dashboard with Plotly Dash

- Plots / Graphs added using Plotly Dash
  - Scatter Plot
  - Pie Chart
- Pie chart displays the total successful launch percentage for each site if All sites option is selected in the dropdown. It displays successful and failure launch percentage if individual lauch site location is selected in the dropdown.
- Scatter chart displays the correlation between the total payload mass and successful launches based on the site location dropdown and payload mass range slider inputs.
- Github repo - Plotly Dash

# Predictive Analysis (Classification)

- **Model Development Pipeline**
  - **Data Preparation**

    Standardize Features → StandardScaler().fit_transform(X) (normalize feature scales)

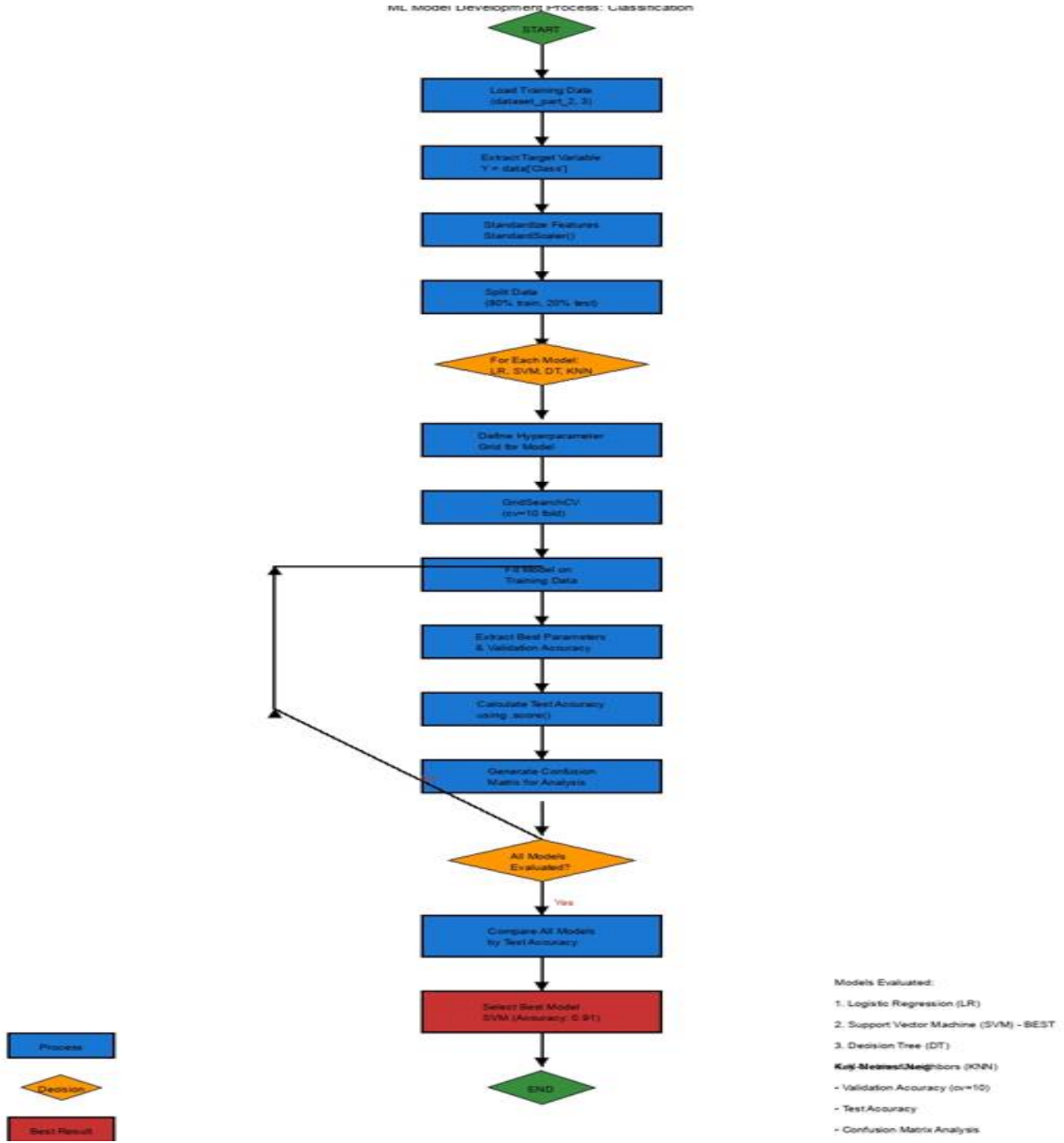
    Train-Test Split → 80% training, 20% testing, random_state=2
  - **Hyper parameter Tuning and Training**

    Define Hyperparameter Grid **,** GridSearchCV with cv=10, Fit Model on Training Data,Extract Best Parameters,,Calculate Validation Accuracy
  - **Model Evaluation and Comparison**
  - **Model Selection**

    SVM (Support Vector Machine) with 0.91 test accuracy .SVM outperformed other models in distinguishing landing outcomes

Repo link -Predictive analysis using cross validation and hyper param tuning

# Predictive Analysis (Classification)



# Results

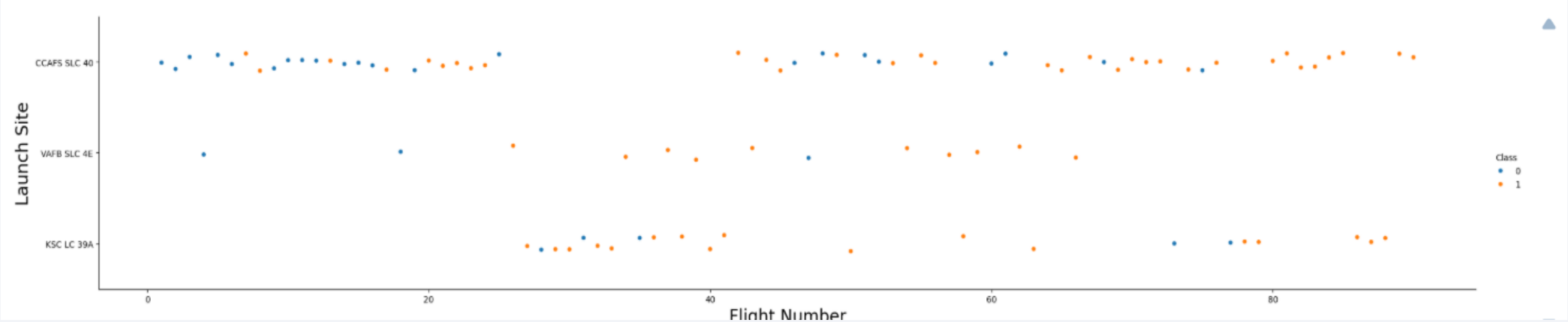
- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Section 2

# Insights drawn from EDA

# Flight Number vs. Launch Site

- Scatter plot of Flight Number vs. Launch Site

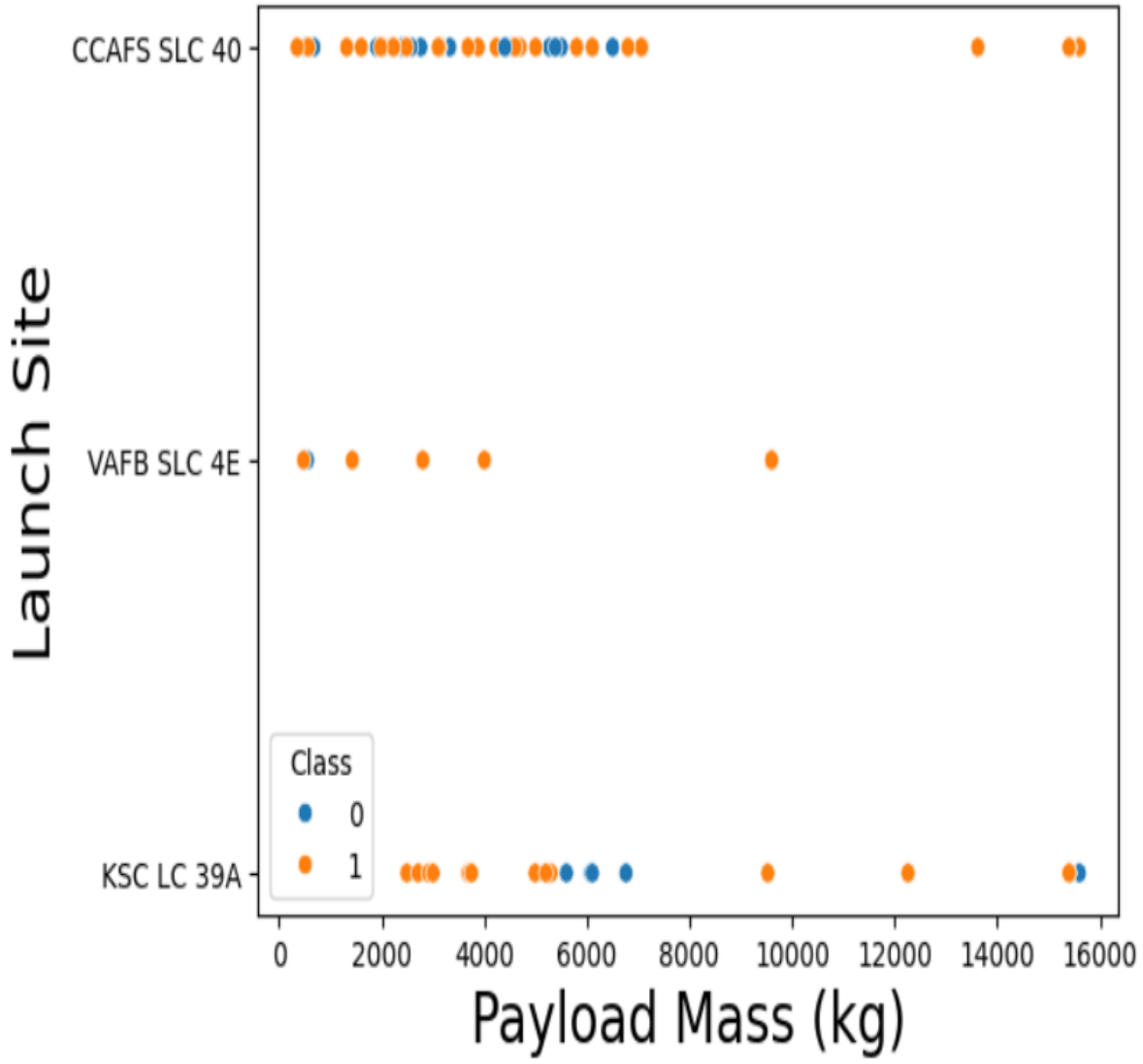



- Scatter plot displays the launch sites on y – axes and flight number on x-axes for success and failure classification.
- It is noted that there are more number of success classification as flight number is increased

# Payload vs. Launch Site

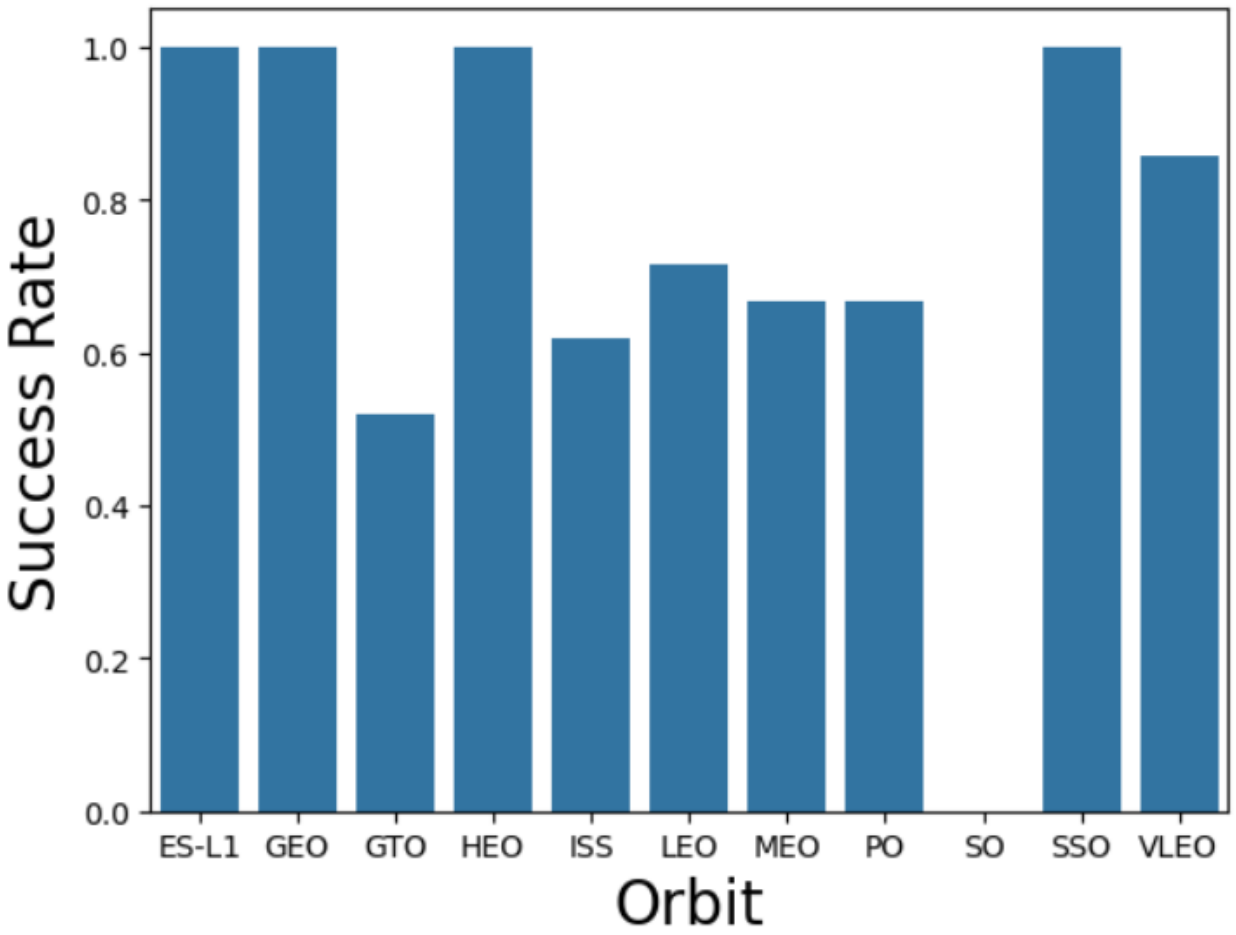
**Scatter plot of Payload vs. Launch Site**

- This plot categorizes each launch sites along with mass of the payload for each lauch. The color code with orange represents successful landing whereas blue represent failure landing.
- It is noted that most of the payload mass are less than 10000 Kg .
- The launch site VAFB SLC 4E has the maximim successful landing rate.
- Launch Site CCAFS SLC 40 has got most of payload masses within 8000 Kg.
- .Successful landing rate is higher for payload mass > 10000 Kg



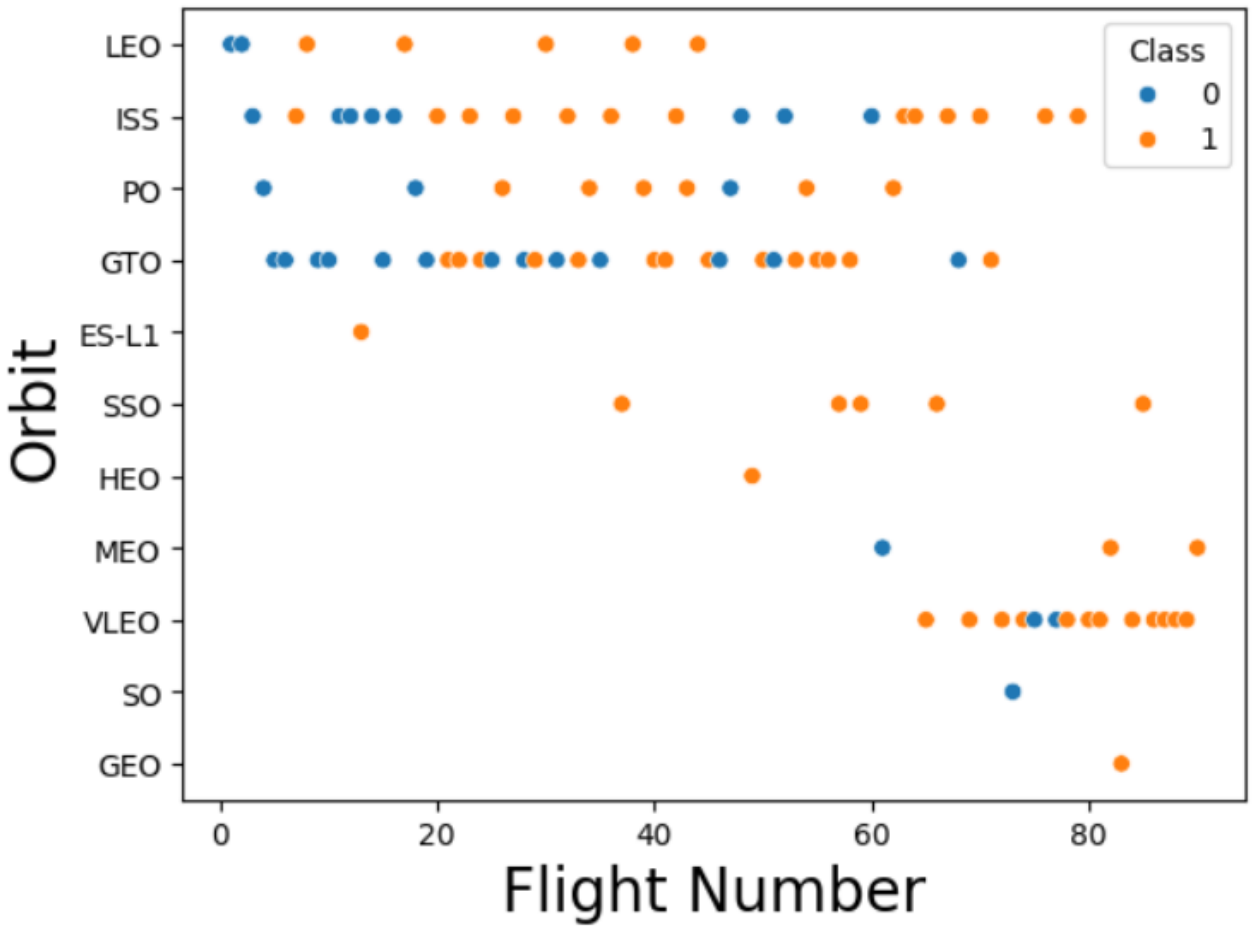
# Success Rate vs. Orbit Type

- This plot represents Orbit on X- axes and Success rate on y-axes
- The Orbit types ES-L1, GEO ,HEO ,SSO has 100% Success Rates
- Success Rates are lower than 60% for SO, ISS and GTO orbit types



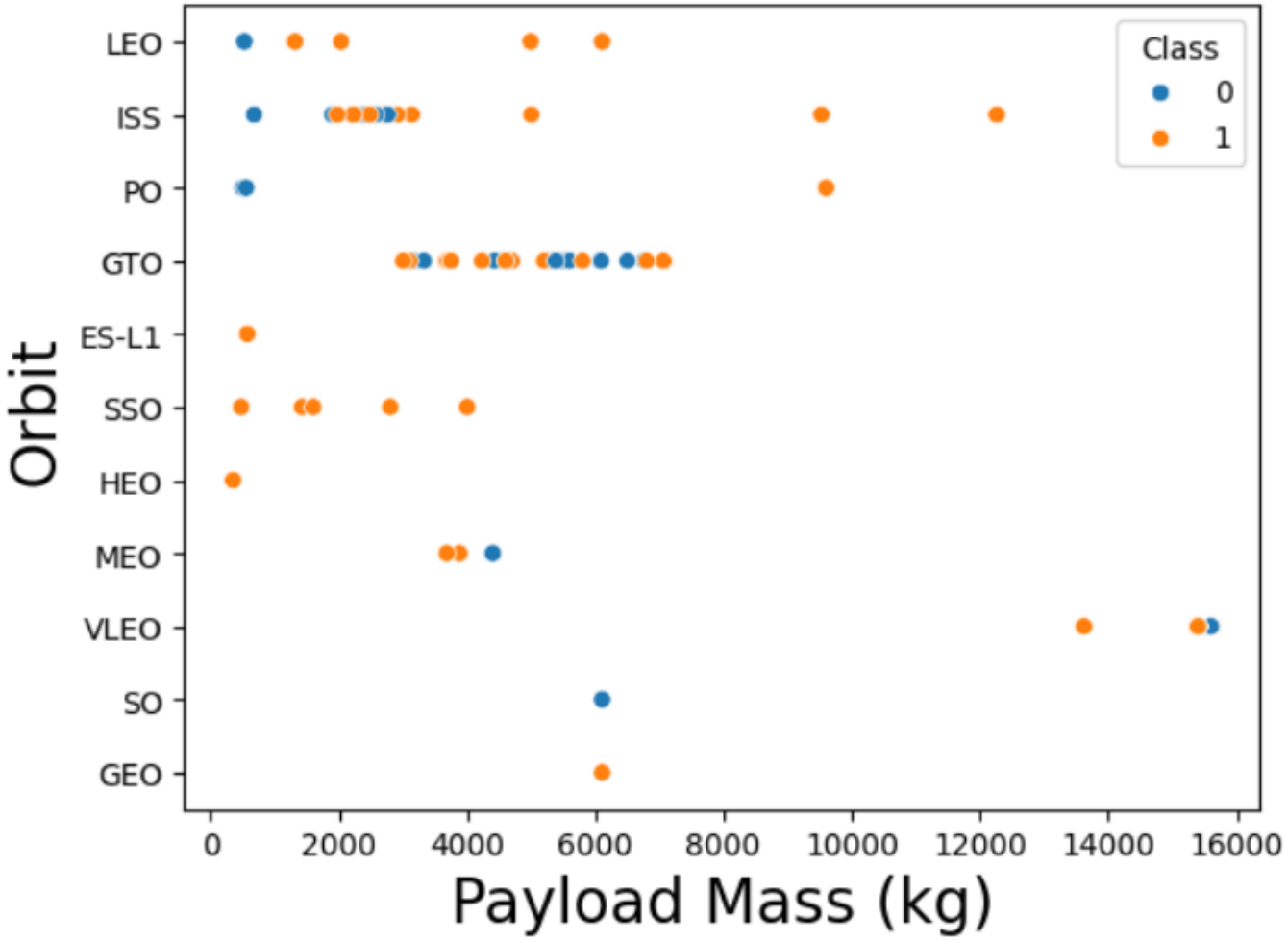
# Flight Number vs. Orbit Type

- Success Rate of Landing increases as Flight Number increases irrespective of Orbit Types.
- There are more launches in ISS and GTO orbit Types and fewer launches in GEO HEO and SO Orbit Types



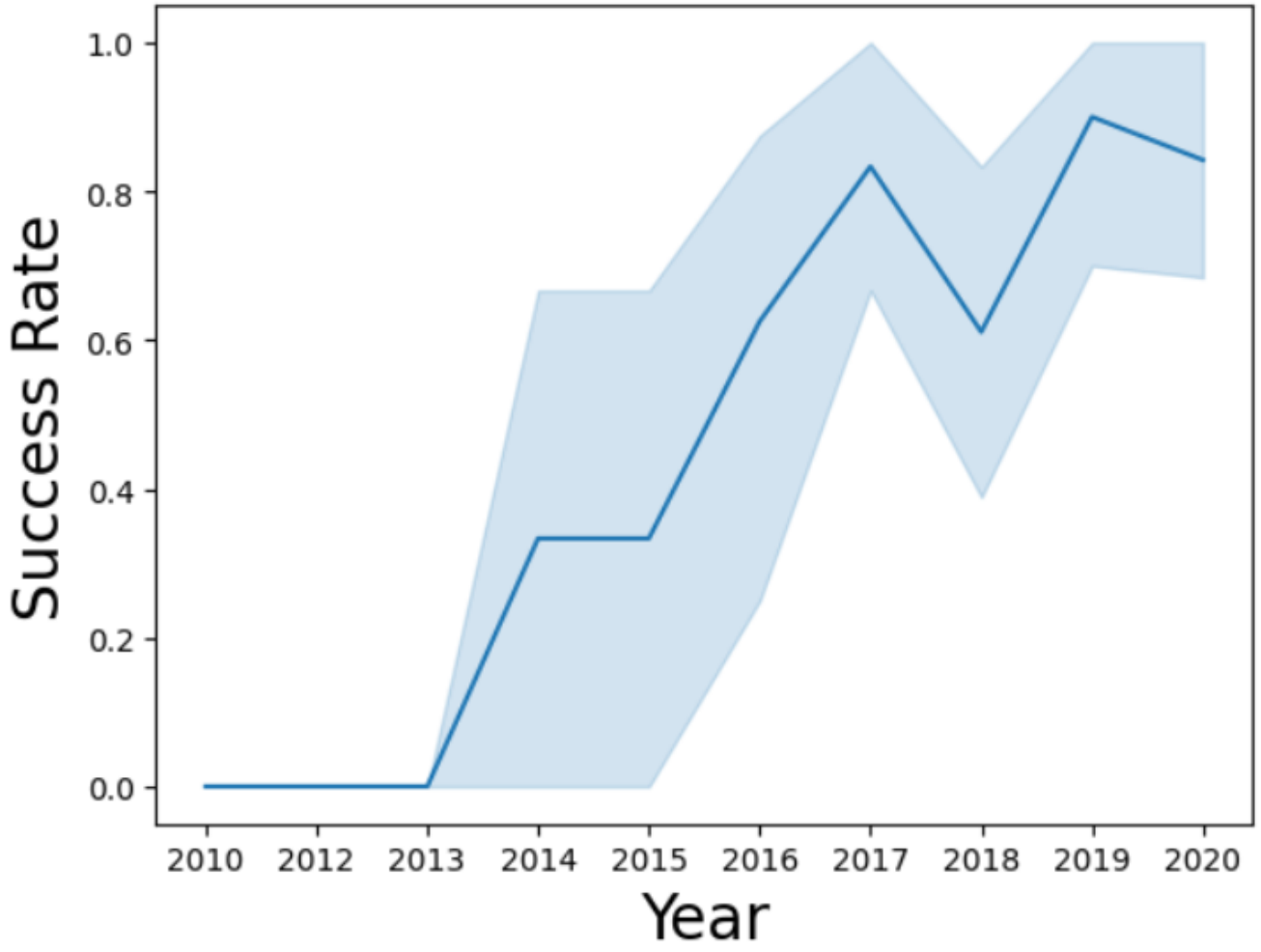
# Payload vs. Orbit Type

- Payload Mass for LEO ISS GTO SSO SO GEO Orbit Types are typically lesser than 8000 Kgs
- Payload Mass > 10000 Kgs have more Success Landing Rate.



# Launch Success Yearly Trend

- ✓ Success rate is spiked after the year 2013. Before 2013 there were no sucessful launch
- ✓ From year 2015 to 2017 there was a significant jump in Launch Success rate although there is a slight drop the following year in 2018.



# All Launch Site Names

- Find the names of the unique launch sites

```
%sql select distinct "Launch_Site" from SPACEXTABLE
```

- SQL Query fetches the unique values from Launch Site column in SPACEXTABL table by using the SQL keyword **distinct .**

| Launch_Site |
| --- |
| CCAFS LC-40 |
| VAFB SLC-4E |
| KSC LC-39A |
| CCAFS SLC-40 |

# Launch Site Names Begin with 'CCA'

```
%sql select * from SPACEXTABLE where
"Launch_Site" like CCA%' limit 5
```

- Query that fetches top 5 records that filters Launch Site column containing the beginning characters of "CCA" using the **like** keyword.

| Date | Time (UTC) | Booster_Version | Launch_Site | Payload | PAYLOAD_MASS__KG_ | Orbit | Customer | Mission_Outcome | Landing_Outcome |
|---|---|---|---|---|---|---|---|---|---|
| 2010-06-04 | 18:45:00 | F9 v1.0 B0003 | CCAFS LC-40 | Dragon Spacecraft Qualification Unit | 0 | LEO | SpaceX | Success | Failure (parachute) |
| 2010-12-08 | 15:43:00 | F9 v1.0 B0004 | CCAFS LC-40 | Dragon demo flight C1, two CubeSats, barrel of Brouere cheese | 0 | LEO (ISS) | NASA (COTS) NRO | Success | Failure (parachute) |
| 2012-05-22 | 7:44:00 | F9 v1.0 B0005 | CCAFS LC-40 | Dragon demo flight C2 | 525 | LEO (ISS) | NASA (COTS) | Success | No attempt |
| 2012-10-08 | 0:35:00 | F9 v1.0 B0006 | CCAFS LC-40 | SpaceX CRS-1 | 500 | LEO (ISS) | NASA (CRS) | Success | No attempt |
| 2013-03-01 | 15:10:00 | F9 v1.0 B0007 | CCAFS LC-40 | SpaceX CRS-2 | 677 | LEO (ISS) | NASA (CRS) | Success | No attempt |

# Total Payload Mass

```
%sql select Customer,
sum("PAYLOAD_MASS__KG_") from
SPACEXTABLE group by Customer having
Customer = 'NASA (CRS)'
```

- ✓ Query calculates the total sum of PAYLOAD_MASS__KG column for 'NASA (CRS)' Customer type.
- ✓ USes the **SUM** function to calculate summation and **Group By** clause for grouping by Customer coumn

| Customer | sum(PAYLOAD_MASS__KG_) |
|---|---|
| NASA (CRS) | 45596 |

# Average Payload Mass by F9 v1.1

- ✓Query that calculates the average payload mass for the booster version F9 v1.1
- ✓Uses the avg function to calculate the average and group by clause to categorize by booster version and having clause to filter by F9 v1.1

```
%sql select "Booster_Version", avg("PAYLOAD_MASS__KG_") from SPACEXTABLE
group by "Booster_Version" having "Booster_Version" = 'F9 v1.1'
```

| Booster_Version | avg(PAYLOAD_MASS__KG_) |
|---|---|
| F9 v1.1 | 2928.4 |

# First Successful Ground Landing Date

```
%sql select min("Date") as
First_Successful_Landing from
SPACEXTABLE where Date in
(select Date from spacextable
where "Landing_Outcome" =
'Success (ground pad)')
```

| First_Successful_Landing |
| --- |
| 2015-12-22 |

- Uses the **min** function to calculate the first landing date
- Uses the **inner query** to filter to filter the landing outcome for Success (ground pad)

# Successful Drone Ship Landing with Payload between 4000 and 6000

**%sql select distinct "Booster_Version" from SPACEXTABLE where "Landing_Outcome" = 'Success (drone ship)' and "PAYLOAD_MASS__KG_" > 4000 and "PAYLOAD_MASS__KG_" < 6000**

- Uses the distinct clause to select the unique booster versions
- Uses the where clause to filter the landing outcome and payload mass column values

| Booster_Version |
|---|
| F9 FT B1022 |
| F9 FT B1026 |
| F9 FT B1021.2 |
| F9 FT B1031.2 |

# Total Number of Successful and Failure Mission Outcomes

**%sql select tb1.success_count , tb2.failure_count from (select count(*) as success_count from SPACEXTABLE Where Mission_Outcome like '%Success%') as tb1, (select count(*) as failure_count from SPACEXTABLE Where Mission_Outcome like '%Failure%') as tb2**

| success_count | failure_count |
|---|---|
| 100 | 1 |

Uses the join on tb1 and tb2 that queries success and failure landing outcomes count respectively and then query the success and failure count from the inner query outcomes

# Boosters Carried Maximum Payload

| Booster_Version | max_payload_mass | count(*) |
|---|---|---|
| F9 v1.1 | 4535 | 5 |
| F9 v1.1 B1018 | 1952 | 1 |
| F9 v1.1 B1017 | 553 | 1 |
| F9 v1.1 B1016 | 4707 | 1 |
| F9 v1.1 B1015 | 1898 | 1 |
| F9 v1.1 B1014 | 4159 | 1 |
| F9 v1.1 B1013 | 570 | 1 |
| F9 v1.1 B1012 | 2395 | 1 |

**%sql select Booster_Version, max("PAYLOAD_MASS__KG_") as max_payload_mass, count(*) from SPACEXTABLE group by Booster_Version having Booster_Version like 'F%' order by count(*) desc**

- Uses the max function to calculate maximum payload mass
- Uses the Group By clause to categorize byeach booster type

# 2015 Launch Records

- List the failed landing_outcomes in drone ship, their booster versions, and launch site names for in year 2015

```
%sql select substr("Date",6,2) as month, "Landing_Outcome", count(*) as count from SPACEXTABLE where substr("Date",0,5) = '2015' group by "Landing_Outcome" having "Landing_Outcome" like '%drone%' order by month
```

- Uses the substr function to fetch month and year from the date column
- Uses count function to calculate count of failures
- Results in 2 failure out comes for drone ship for the Month of January in the year 2015

| month | Landing_Outcome | count |
|---|---|---|
| 01 | Failure (drone ship) | 2 |
| 06 | Precluded (drone ship) | 1 |

| Landing_Outcome | count |
|---|---|
| No attempt | 10 |
| Success (drone ship) | 5 |
| Failure (drone ship) | 5 |
| Success (ground pad) | 3 |
| Controlled (ocean) | 3 |
| Uncontrolled (ocean) | 2 |
| Failure (parachute) | 2 |
| Precluded (drone ship) | 1 |

# Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

```
%sql select  "Landing_Outcome", count(*) as
  count from SPACEXTABLE where Date BETWEEN
  '2010-06-04' AND '2017-03-20' group by
  "Landing_Outcome" order by count desc
```

- Uses the BETWEEN Clause to filter between specified date ranges
- Uses the group by clause for landing outcome column and calculates the count of each outcomes
- Uses Order by Desc to rank the count in descending order from highest to lowest

Section 3

# Launch Sites Proximities Analysis

# Mark all Launch Sites on the Map

**Create the initial Map**

```
site_map = folium.Map(location=nasa_coordinate, zoom_start=5)
```

**Mark all the Sites with circles and markers**

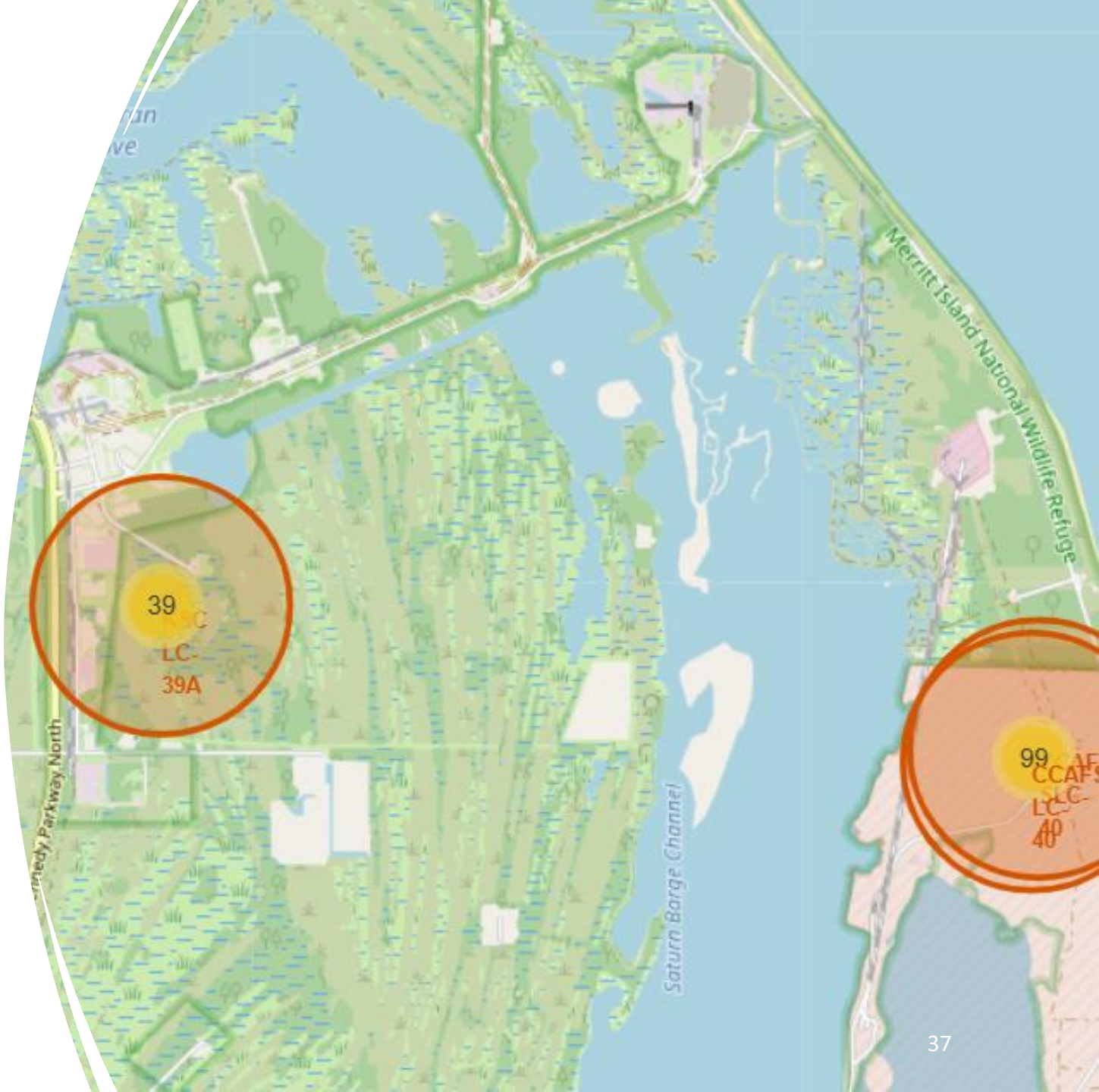
```
site_map.add_child(circle)
site_map.add_child(marker)
```

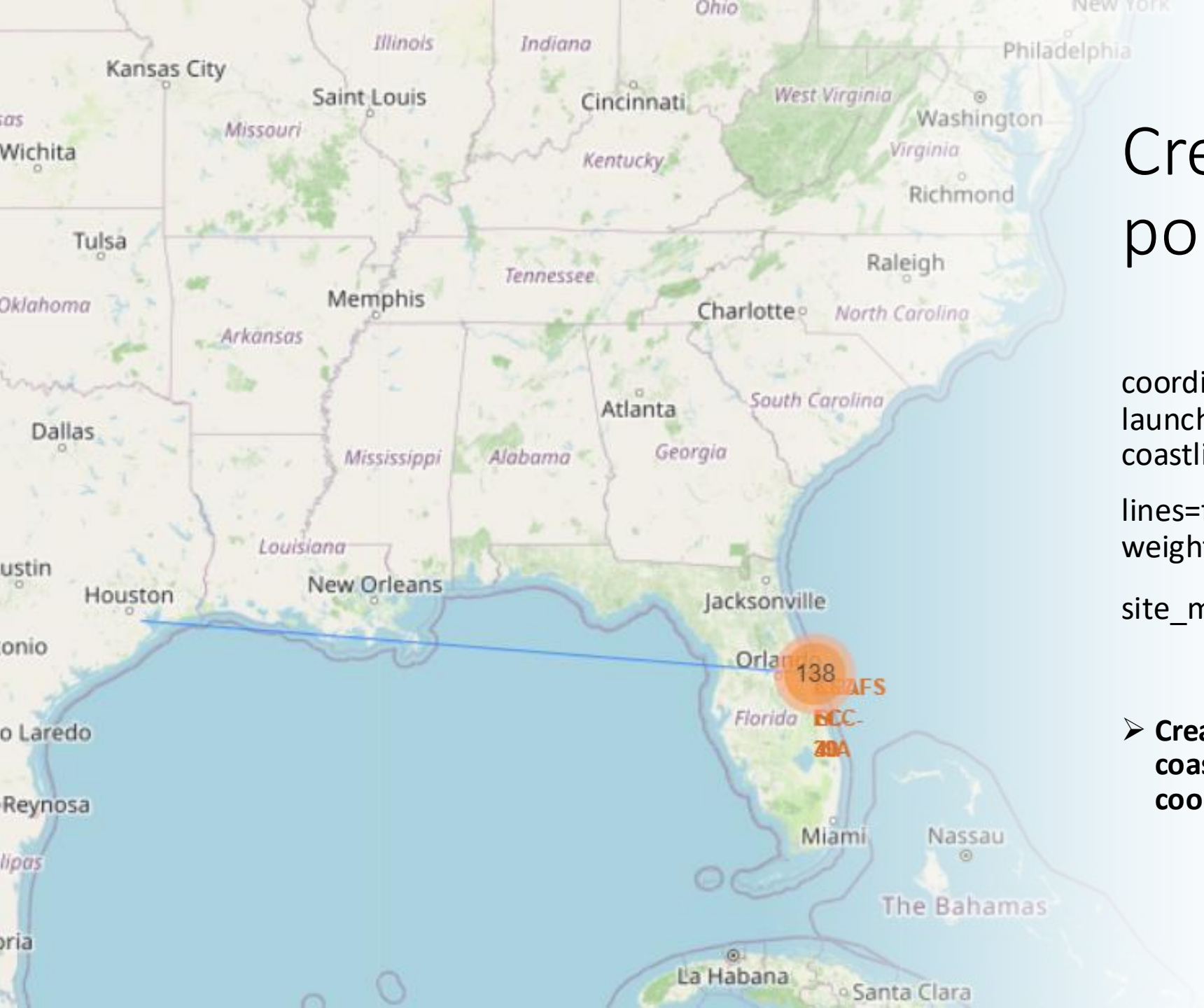
# Add Marker Cluster to folium Map Object

```
marker_cluster = MarkerCluster()
marker =
folium.Marker(location=[record['Lat'],
record['Long']],
icon=folium.Icon(color='white',
icon_color=record['marker_color']))
    marker_cluster.add_child(marker)
```

site_map.add_child(marker_cluster)

- ✓ Adds the marker Cluster and markers to the folium Map Object






# Create a Folium polyline Object

coordinates = [[launch_site_lat, launch_site_lon], [coastline_lat, coastline_lon]]

lines=folium.PolyLine(locations=coordinates, weight=1)

site_map.add_child(lines)

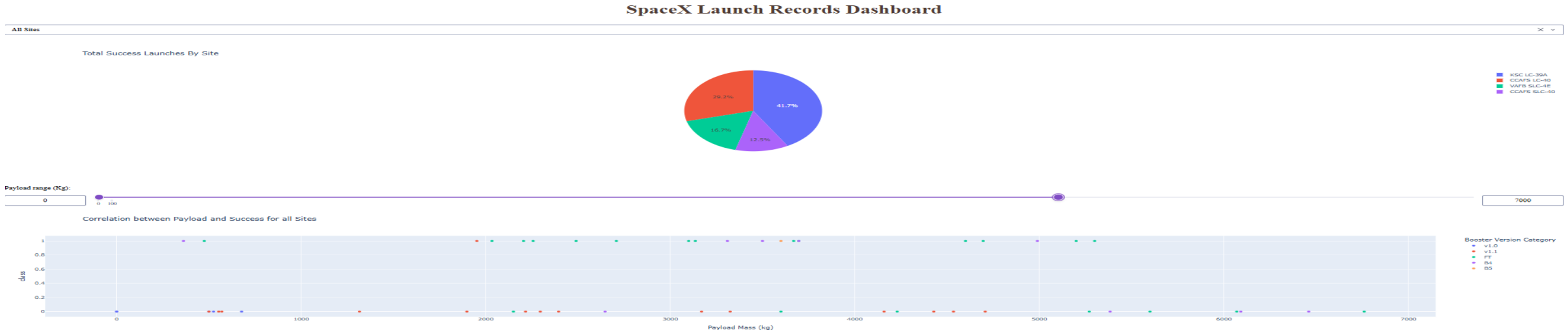
- **Create a `folium.PolyLine` objectusing the coastline coordinates andlaunch site coordinate**

Section 4

# Build a Dashboard with Plotly Dash

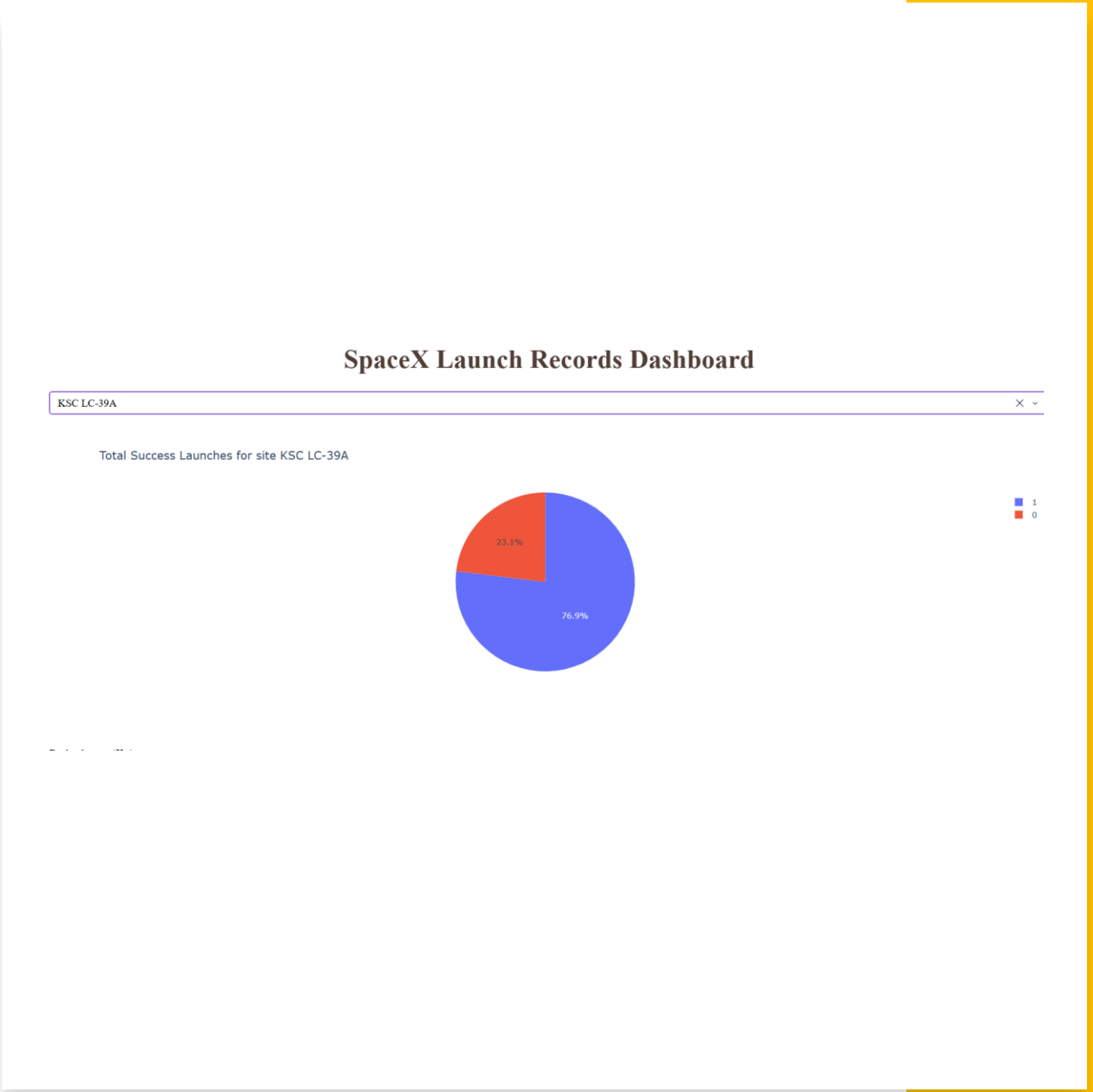
# SpaceX Launches Dashboard

- Plotly Dash Library is used to render dash core components drop down and range slider
- A decorated call back function is used to display pie chart and scatter plots based on the input from dropdown and range slider.
- When All site is selected pie chart displays success landing rate on en each launch sites.
- Range slider is used to adjust the filter of the payload mass reflecting the scatter plots plotted within that range



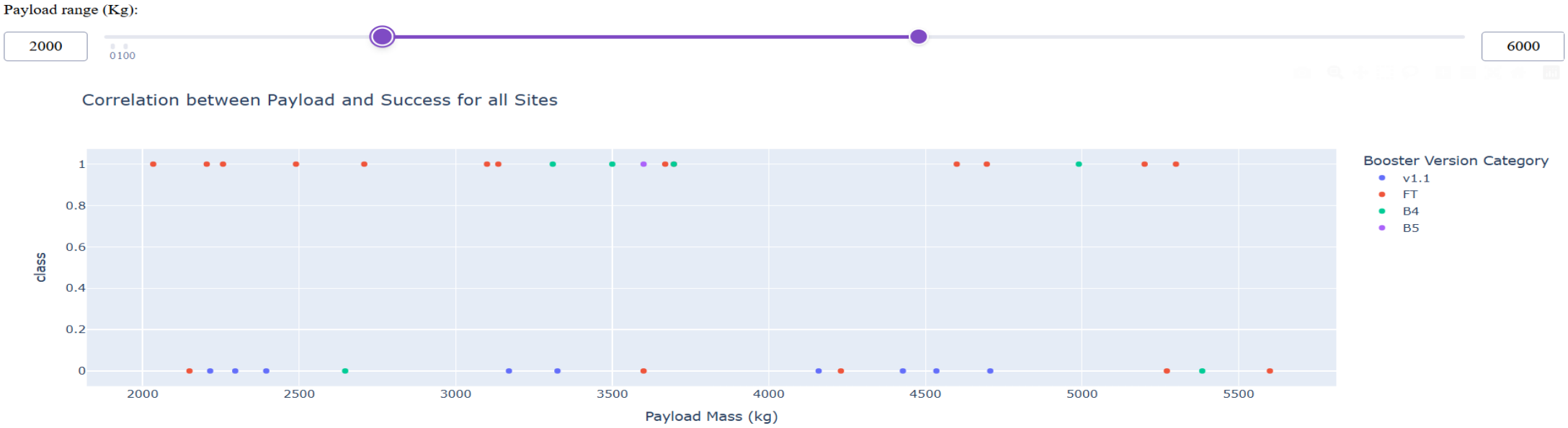
# Pie Chart display for specific Launch Site

- When Specific Launch site is selected from the drop down success and failure landing rates represented as class 1 and classs 0 lables respectively is depicted in the pie chart



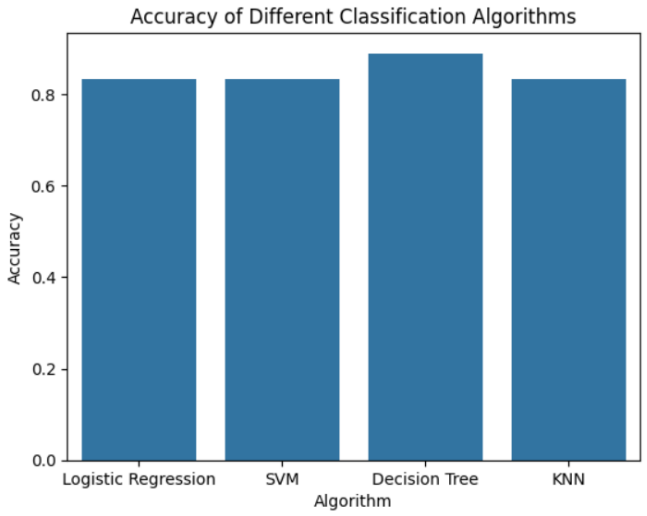
# Scatter Plots - Payload Mass vs Class

- Scatter Plot distribution varies based on the inputs driven from range slider and dropdown
- Callback function that is decorated with input and output paramater is used to render the scatter plots
- Range slider input is adjusted to filter the payload mass within the specific range. Whenever the range input changes scatter plot is re rendered automatically with the new input



Section 5

# Predictive Analysis (Classification)



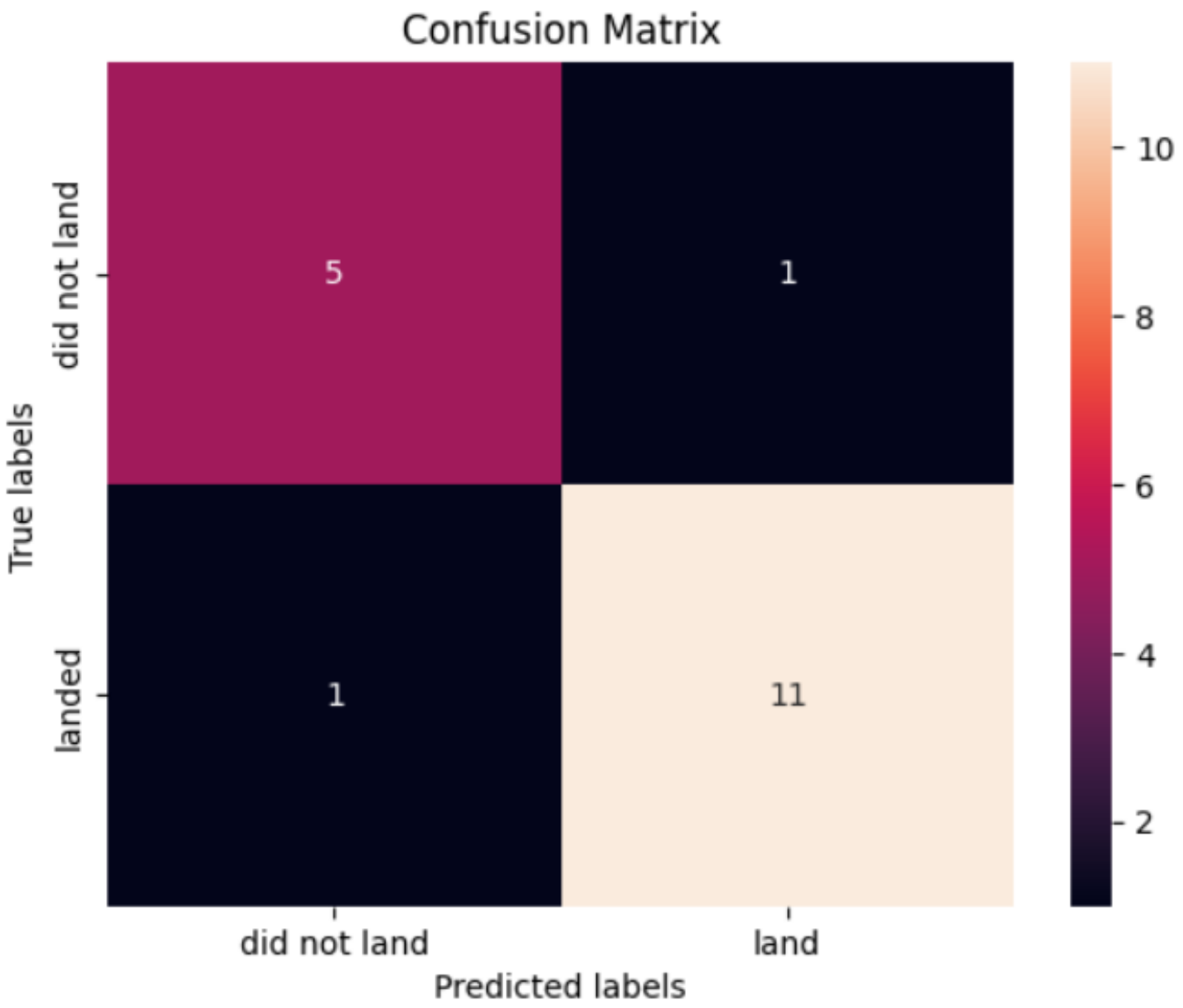

# Classification Accuracy

- Decision Tree classifier algorithm obtained the highest accuracy of around 89%
- All models Logistic Regression, SVM, Decision Tree and KNN relatively performed well achieving the accuracy score of 80% or more.

# Confusion Matrix

- Decision Tree classifier algorithm predicted 16 out of 18 samples correctly
- Number of True Positives : 11
- Number of True Negatives : 5
- Number of False Positives : 1
- Number of False Negatives : 1

```
tree_cv.predict(X_test)
confusion_matrix(Y_test,yhat)
```



# Conclusions

**1. 📊 "Data Collection: Diversity of Sources Strengthens Insights"**
**Principle:** Multi-source data integration (APIs, web scraping, CSV files, SQL databases) creates richer datasets than single sources—combining different collection methods reveals patterns invisible in isolated datasets, enabling more robust analysis.
**2. 🔧 "Data Wrangling: The Foundation for Success"**
**Principle:** 80% of data science effort goes to cleaning, standardization, and feature engineering—quality preprocessing (handling missing values, standardizing scales, binary classification) directly determines model performance more than any algorithm choice.
**3. 📈 "Exploratory Data Analysis: Insights Before Models"**
**Principle:** Visualization and statistical exploration (scatter plots, distributions, correlations, temporal trends) uncover relationships and guide feature selection—EDA prevents wasted effort on irrelevant models and focuses analysis on what matters.
**4. 🎯 "Model Evaluation: Comparison > Single Model"**
**Principle:** Testing multiple algorithms (Logistic Regression, SVM, Decision Trees, KNN) with rigorous validation (cross-validation, train-test split, confusion matrices) identifies the **best performer**—no single model wins universally; empirical comparison is essential.

# Appendix

- **13 Sections** with detailed code snippets
- **10 SQL Queries** with results
- **5 EDA Visualizations** documented
- **4 ML Models** compared
- **All Asset Files** catalogued

# Thank you!